Discount Optimizer

The Complete System Guide

How the engine decides the right selling price for every product in every city, every week - the full workflow, the logic behind each stage, the proof it works, and how to read every output.

Finds discount that is WASTED (customers would buy anyway)

Finds discount that WORKS (real, un-borrowed volume growth)

Refuses to guess where the data is too thin

Proves itself on data it has never seen - receipts, not promises

Reference deployment: 4 SKUs x 11 cities on Blinkit | regenerate: scripts/generate_system_guide.py

Contents

1. The business problem it solves

A CPG brand on quick-commerce spends lakhs every month on discounts. Some of that discount genuinely creates sales. A large share does not - customers would have bought at a higher price, and the difference is margin given away for nothing. The two are invisible to the naked eye because they are mixed together in the same sales numbers, and they differ by product AND by city.

The question the system answers, every Monday:

"For each product, in each city, what selling price should be live this week - so we stop paying for discount customers don't notice, and redirect it to where it genuinely grows volume?"

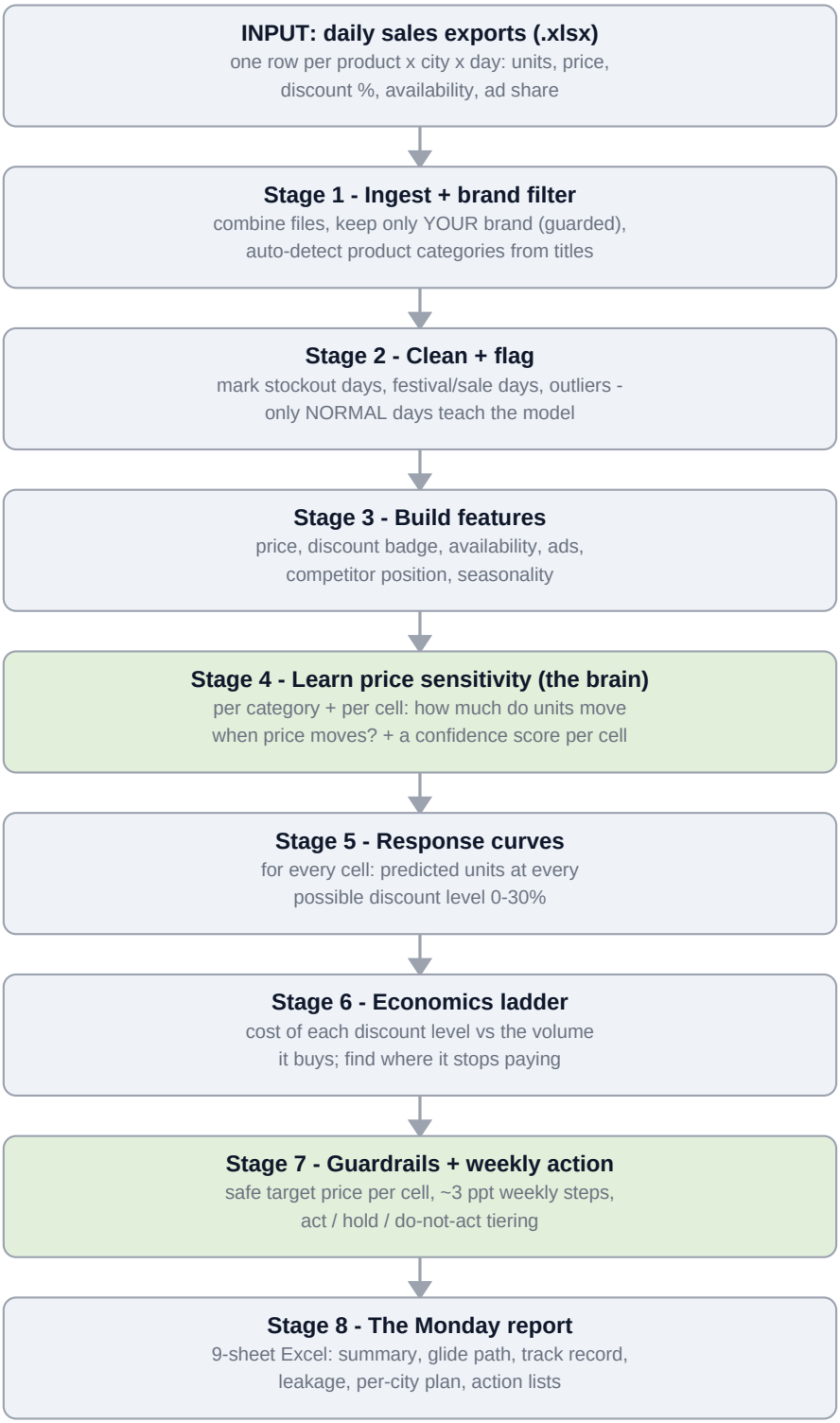
Reference numbers (live deployment)

Metric	Value
Portfolio	4 SKUs x 11 cities = 33 product-city cells
Gross sales	Rs. 78.9 L / month
Discount spend	Rs. 18.4 L / month (23.3% of gross)
Recoverable waste identified	~ Rs. 1.76 L / month across 29 cells

Numbers regenerate on every weekly run; these are from the current live report. A 30-50 SKU brand carries proportionally larger discount spend and waste.

2. The system at a glance

One year of daily platform data goes in; one 9-sheet Excel action plan comes out. Eight stages run in sequence - each stage's output is the next stage's input:



Around the pipeline sits a proof layer (Section 5): a readiness gate before any new brand is onboarded, and three validation documents that regenerate from code on every run.

3. Stage-by-stage: what each step does and why

Stage 1 - Ingest and brand filter

- Reads every Excel export, combines and de-duplicates them, and keeps only YOUR brand's rows (competitor rows are kept aside solely to measure your price position against them).
- Brand matching is guarded: if the configured brand name matches nothing, or accidentally catches a competitor's name, the run STOPS with a clear message instead of silently modelling the wrong rows.
- Product categories are detected automatically from product titles (brand and pack-size words stripped) - so a new brand onboards without hand-written keyword lists.

Stage 2 - Clean and flag (why: garbage in, garbage out)

- Every day is labelled: stockout day (can't sell what isn't there), festival/platform-sale day (demand spike has nothing to do with your price), or statistical outlier (a weird spike/dip).
- Only the remaining NORMAL days train the model. This single discipline is why the learned price sensitivity reflects price - not Diwali.

Stage 3 - Build features (why: price is not the only lever)

- For each cell and day the system assembles what else could explain sales: availability, advertising share, competitor price position, weekday/season. The model must credit those factors first - whatever remains attributable to price is the true price effect.

Stage 4 - Learn price sensitivity (the brain)

- For each category, a robust regression learns: when THIS cell's price moved, holding ads/availability/season constant, how much did units move? That slope is the ELASTICITY (e.g. -1.8 = a 1% price cut adds ~1.8% volume).
- Each cell gets its own elasticity, stabilised toward its category's typical value when its own history is thin - plus a 0-100 CONFIDENCE SCORE built from data depth, price variation, fit quality, plausibility, and statistical tightness.
- Cells scoring too low are marked DO NOT ACT. The system would rather say 'not enough evidence' than invent a number.

Stage 5 - Response curves

- Using the learned sensitivity, the system sweeps every discount level 0-30% and predicts the units each level would sell - one curve per cell. This is the menu of choices the economics stage prices out.

Stage 6 - Economics ladder

- For each discount level: what does it cost in discount spend, and what volume does it buy? Marginal logic finds the ELBOW - the depth where one more point of discount stops paying for itself.

Stage 7 - Guardrails and the weekly action (why: safety over theory)

- Target price = the cell's HISTORICAL FLOOR: the lowest discount the cell has actually operated at recently (its proven-safe level) - never a price customers have never seen.
- Moves glide ~3 percentage points per week, so no city gets a price shock and each week's response is observed before the next step.
- Every cell is tiered: STRONG CUT (clear win, act), TRADE-OFF (positive but review), HOLD, INCREASE (rare), or DO NOT ACT / NEEDS TEST (confidence too low - run a small price test first).
- Two extra gates: cells with weak sensitivity (inelastic, |e| <= 1) are flagged 'discount unlikely to pay - hold or raise'; and any 'growth' that the leakage check shows is borrowed or stolen is removed before a cell can qualify for deeper discount.

Stage 8 - The Monday report

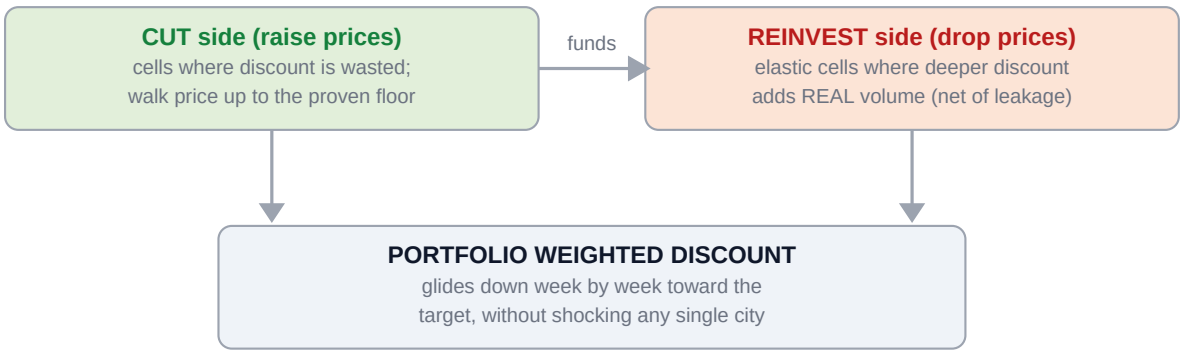
- Everything lands in one Excel workbook the brand team reads in ten minutes and executes in one platform session. Every number is a live formula - edit an assumption and the sheets recompute.

Worked example - one cell end to end

Jaggery Powder in Delhi-NCR: the model learns the cell is only mildly price-sensitive; its proven floor is a much smaller discount than it currently runs. The plan walks the price up ~3 points per week over a month to the floor, projects the small volume dip that history supports, and books the discount spend saved - visible as one row on the By Product sheet with its exact weekly prices.

4. The decision logic: cut, reinvest, hold

The portfolio runs as a flywheel: money recovered from wasted discount funds the few places where deeper discount genuinely grows the business.



The leakage check - real vs borrowed vs stolen

Before any cell qualifies for deeper discount, its past promo bumps are decomposed. A bump is not all new demand:

A promo week's unit spike, decomposed:

REAL: kept. BORROWED: post-promo dip - customers stocked up, sales taken from next weeks. STOLEN: your own sibling pack dipped while this one spiked.

Only REAL demand counts toward a 'deeper discount works here' decision.

Observational estimates - directional signals, not controlled experiments; stated as such wherever they appear. On the reference brand, most staples showed low leakage, while Sunflower Oil showed 12-18% borrowed volume in its worst cities (edible oil stores well - customers stockpile).

The safety gates, in order

Gate	Rule	Why
Confidence gate	Model confidence too low -> DO NOT ACT / Needs Test	never act on thin or contradictory data
Inelastic gate	|elasticity| <= 1 -> 'discount unlikely to pay'	volume response can't cover the subsidy; hold or raise
Leakage haircut	growth counted net of borrowed + stolen units	stockpiling and pack-swapping are not growth
Historical floor	never target a price the cell hasn't survived	no experiments on live revenue
Glide rule	~3 ppt per week, observed before the next step	no price shocks; every move is reversible

5. The proof layer: why the numbers can be trusted

The system does not ask to be trusted - it ships three validation documents that regenerate from code on every run, plus a readiness gate for any new brand's data.

Proof	What it does	Current result
Credibility report	Separates the honest accuracy of the price engine (the part that actually sets prices) from the inflated fit of the full statistical model, on held-out data.	price-engine R-squared ~0.87 at the decision grain, ~26% error - self-rated 'Moderate', published, not hidden
Track record / forward test	Trains on old data only, then grades predictions on 8 later weeks the model never saw - including: when price actually rose, did volume fall as predicted?	Direction correct and CONSERVATIVE: predicted -13.8% volume where reality was -8.8%. Following it is safer than it claims.
Recovery test	Plants a KNOWN price sensitivity in synthetic data with a deliberate trap (discounts co-timed with ad spikes), then checks the pipeline finds the planted answer.	Found it ~3.6x closer to truth than a naive spreadsheet fit; small honest residual (~0.2) reported, not hidden
Readiness gate	Before any new brand is onboarded: a one-page verdict on what share of their portfolio can be acted on with confidence, per product and city.	GREEN / YELLOW / RED verdict; RED = do not run production - design a price test instead

Also standing guard: 32 automated tests run before any change ships, and the input validator stops a run loudly if a new data file is missing columns or unusable.

6. How to read the outputs

The weekly deliverable is WASTE_REINVEST_REPORT.xlsx - nine sheets in reading order. Ten minutes, top to bottom:

Sheet	What it shows	How to act on it
1. Summary	The portfolio in one screen: gross sales, discount spend, net revenue - today vs after the plan. Plus this week's move counts and the honest model-accuracy block.	Read first. If discount spend 'after cuts' is meaningfully lower while units barely move, the week is working.
2. Glide Path	Week-by-week projection: portfolio weighted discount, spend and cumulative savings for each coming week.	Use it to see when the plan completes and what the end-state saves per month.
3. Track Record	Part A: the forward test (trained blind, graded on unseen weeks) with its conservative verdict. Part B: predicted vs actual per city - fills with YOUR results once you act.	This is the receipts sheet. Per-city rows are noisy by nature; the reliable signal is the aggregate direction in Part A.
4. Leakage	Per cell: how much of past promo lift was real vs borrowed (phi) vs stolen (kappa), plus the 'worth discounting?' verdict (inelastic cells flagged).	Cells marked 'unlikely to pay' -> hold or raise. High borrowed% -> discount that product shallower.
5. By Product (the workhorse)	For each product: a city-by-city table - confidence, current price, target price, action, savings, and the exact selling price for every coming week (W1, W2, ...).	This is what the ops team executes. Set this week's column on the platform; done.
6. Price Lifts	Every 'raise price' move as a flat list, sorted by money wasted per month.	Bulk-execution list for the platform panel; biggest savings first.
7. Price Drops	The few strategic 'discount deeper' moves that passed every gate (elastic, real demand, budget-positive).	Small list by design. If it's empty, nothing qualified - that is a finding, not a bug.
8. Needs Test	Cells the model refuses to act on (thin or contradictory data) with what a small price test would resolve.	Pick 1-2 per month for a 2-4 week mini-test; they graduate into the plan once data supports them.
9. Data (hidden)	The raw per-cell numbers every other sheet's formulas reference.	Unhide to audit any figure; edit a value to see the plan recompute.

The proof documents (share with any skeptic)

File	Read it as
v4_outputs/_readiness/DATA_READINESS_REPORT.md	'What share of this brand's portfolio can be acted on today?' - the onboarding verdict
v4_outputs/_credibility/CREDIBILITY_REPORT.md	'How accurate is the engine that actually sets prices?' - the honest R-squared, plus bias checks
v4_outputs/_proof_loop/PROOF_LOOP_REPORT.md	'Did its forecasts come true on weeks it never saw?' - the forward test and conservative verdict
v4_outputs/_recovery/RECOVERY_REPORT.md	'Can it find an answer we planted?' - the machinery test a data scientist will respect

Also written each run: recommendations.csv (every cell's full decision detail), waste.csv / reinvest.csv (the two action lists in raw form), and outliers_removed.csv (audit trail of excluded days).

7. The weekly operating rhythm

When	What happens	Who
Monday AM	Fresh data in; double-click run.bat; report opens	operator
Monday AM	10-minute read: Summary -> Track Record -> By Product	brand team
Monday PM	Strong Cut rows approved as-is; Trade-off rows reviewed	brand team
Tue-Wed	Approved prices go live on the platform	brand ops
Thursday	Mid-week sanity check vs prediction	both
Next Mon	New data arrives; model re-learns; Track Record Part B fills with predicted-vs-actual; cycle repeats	

The system is self-correcting: every week's real response feeds the next week's model. Monthly, the readiness report re-runs to confirm the actionable share of the portfolio is growing.

The handful of dials that matter (v4_config.py)

Dial	Default	What it changes
BRAND_NAME	-	the one line to set for a new brand
TARGET_TIMELINE_WEEKS	12	how fast gaps close (8 = aggressive)
MIN_DISCOUNT_CHANGE_PPT	3	smallest weekly price step
HISTORICAL_FLOOR_PERCENTILE	25	how deep the proven-safe floor reaches
INELASTIC_ELASTICITY_THRESHOLD	1.0	the 'discount unlikely to pay' line

8. Honest limits - what this system is NOT

- **Not a demand forecaster.** It predicts how volume responds to PRICE, not next month's total sales. Trend, festivals and demand shocks dominate absolute volume - the forward test shows this plainly, and we say it before anyone asks.
- **Not rupee-precise.** Savings are directionally validated ranges. The forward test shows the engine errs on the safe side - real volume loss was smaller than predicted - so quote ranges, lean on direction.
- **Not causal proof.** Estimates come from observed history, not randomized experiments. The remedy is built in: act, then read Track Record Part B - predicted vs actual on YOUR moves is the strongest proof there is.
- **Not fire-and-forget.** One platform per run, daily data required, and a human approves every move. The system recommends; people decide.
- **Not universal.** Cells with thin history stay behind the Needs-Test gate until a small price test earns them in. The readiness report says upfront how much of a portfolio is actionable.

9. Glossary

Term	Plain meaning
Cell	one product in one city - the unit everything is decided at
Elasticity	how strongly units respond when price moves; -2 means a 1% price cut adds ~2% volume
Inelastic	elasticity weaker than 1: the volume gained can't cover the discount given - discounting can't pay
Historical floor	the lowest discount a cell has actually operated at recently - the proven-safe target
Elbow	the discount depth where one more point stops paying for itself
Glide path	the week-by-week walk from today's price to the target
Pull-forward (phi)	promo sales borrowed from future weeks (stock-up), visible as the dip after the promo
Cannibalization (kappa)	promo sales stolen from your own other pack sizes of the same product
Held-out / forward test	grading the model on data it never saw during learning - the honest way to measure accuracy
Confidence tier	per-cell evidence score deciding whether the system may act (High/Medium) or must wait (Low/Do-not-ac
Needs Test	the system's refusal to guess: run a small controlled price change to generate the missing evidence
Weighted discount	portfolio discount % weighted by sales - the single number the glide path moves down

Generated by scripts/generate_system_guide.py - regenerate after any system change so this document never drifts from the code.